Юніт тести

1)Тестування Food

Тест
```
package org.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class FoodTest {

    @Test
    void testAddFood() {
        Food apple = new Food("Apple", 52);

        assertEquals("Apple", apple.getName(), "Назва продукту повинна бути
'Apple'");
        assertEquals(52, apple.getCalories(), "Калорії повинні дорівнювати 52");
        assertNotNull(apple.getName(), "Назва продукту не повинна бути null");
        assertThrows(IllegalArgumentException.class, () -> new Food("Orange", -
10),"Калорії не можуть бути від'ємними");
    }
}
```

Код
```
package org.example;

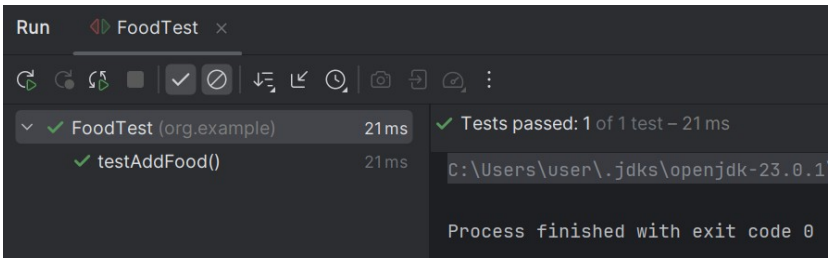
public class Food {
    private String name;
    private int calories;

    public Food(String name, int calories) {
        if (calories < 0) {
            throw new IllegalArgumentException("Калорії не можуть бути
від'ємними");
        }
        this.name = name;
        this.calories = calories;
    }

    public String getName() {
        return name;
    }
```

```
    public int getCalories() {
        return calories;
    }
}
```

Результат



2) Тестування Exercise

Тест

```
package org.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ExerciseTest {

    @Test
    void testAddExercise() {
        Exercise running = new Exercise("Running", 300);

        assertEquals("Running", running.getName(), "Назва вправи має бути
'Running'");
        assertEquals(300, running.getCaloriesBurned(), "Спалені калорії повинні
дорівнювати 300");
        assertTrue(running.getCaloriesBurned() > 0, "Спалені калорії повинні бути
більше нуля");
        assertThrows(IllegalArgumentException.class, () -> new
Exercise("Swimming", -100),"Спалені калорії не можуть бути від'ємними");
    }
}
```

Код

```
package org.example;

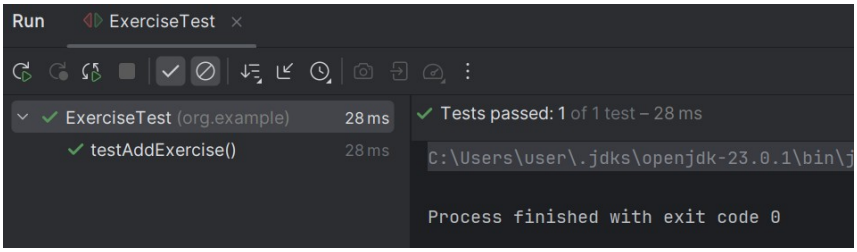
public class Exercise {
    private String name;
    private int caloriesBurned;

    public Exercise(String name, int caloriesBurned) {
        if (caloriesBurned < 0) {
            throw new IllegalArgumentException("Спалені калорії не можуть бути від'ємними");
        }
        this.name = name;
        this.caloriesBurned = caloriesBurned;
    }

    public String getName() {
        return name;
    }

    public int getCaloriesBurned() {
        return caloriesBurned;
    }
}
```

Результат



3)Тестування CalculateCalories

Тест

```
package org.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculateTest {

    @Test
```

```
    void testCalculateTotalCalories() {
        Food apple = new Food("Apple", 52);
        Exercise running = new Exercise("Running", 300);
        CalculateCalories calculator = new CalculateCalories();

        int netCalories = calculator.calculateNetCalories(apple, running);

        assertEquals(0, netCalories, "Залишок калорій повинен бути 0");
        assertFalse(netCalories < 0, "Залишок калорій не може бути від'ємним");
        assertNotNull(calculator.calculateNetCalories(apple, running), "Результат
обчислення не повинен бути null");
        assertDoesNotThrow(() -> calculator.calculateNetCalories(new
Food("Banana", 100), new Exercise("Yoga", 100)),
                "Обчислення калорій не повинно викидати виключення при рівних
значеннях калорій");
    }
}
```

Код

```
package org.example;

public class CalculateCalories {

    public int calculateNetCalories(Food food, Exercise exercise) {
        int netCalories = food.getCalories() - exercise.getCaloriesBurned();
        return Math.max(netCalories, 0); // Залишок калорій не може бути
від'ємним
    }
}
```

Результат

